Innovative Evaluation Practices for BCSE308P - Computer Networks Lab

Fall Semester 2026-27

DESIGN AND IMPLEMENTATION OF A RELIABLE DATA TRANSFER PROTOCOL OVER UDP

24BCE0593	P.Dinesh Reddy
24BCT0291	B.V N D Saran

B.Tech.

in

Computer Science and Engineering

School of Computer Science and Engineering



October 2026

ABSTRACT

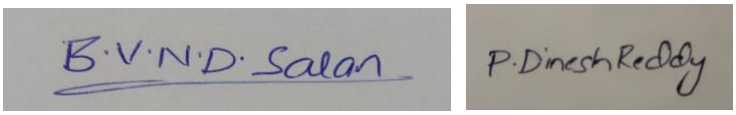
The User Datagram Protocol (UDP) offers fast, connectionless delivery but gives no guarantee that data arrives, arrives only once, arrives intact or arrives in order. This project designs and implements a reliable data transfer (RDT) protocol on top of UDP in Python and evaluates it against a baseline. The protocol adds session set-up (SYN/SYNACK), 32-bit sequence numbers, a CRC-32 checksum, acknowledgements, retransmission timers and a sliding window, followed by a FIN/FINACK teardown.

Three automatic repeat request (ARQ) schemes share one packet format: Stop-and-Wait, used as the baseline; Go-Back-N; and Selective Repeat, the proposed mechanism, which acknowledges every packet individually, keeps a timer per packet, resends only lost packets and buffers out-of-order data at the receiver. A built-in channel emulator injects packet loss, bit corruption, delay and reordering so that experiments are repeatable. Each scenario transferred a 100 KB file ten times, and every received file was verified with SHA-256.

Compared with Stop-and-Wait at 5, 10 and 20 percent loss, Selective Repeat raised mean throughput by 238, 205 and 205 percent and cut transfer time by about 67 to 69 percent. Compared with Go-Back-N it needed about 72 to 76 percent fewer retransmissions, and under packet reordering it sustained 166 KB/s against 20 KB/s. At 5 percent loss, however, Selective Repeat and Go-Back-N were statistically indistinguishable, and the fixed retransmission timeout and the absence of congestion control remain limitations. The work shows how the principles of reliable transport can be built, tested and measured at application level.

Certificate

This is to certify that the project report titled "Design and implementation of a reliable data transfer protocol over UDP" is a bonafide record of the work done by P.Dinesh Reddy (24BCE0593), B.V N D Saran(24BCT0291) in partial fulfilment of the requirements of the Innovative Evaluation Practices for BCSE308P - Computer Networks Lab during the Fall Semester 2026-2027.



Name and Signature of the Students

TABLE OF CONTENTS

Sl.No	Contents	Page No.
	Abstract	2
1.	**INTRODUCTION**	6
	1.1 Background and motivation	6
	1.2 Problem statement	6
	1.3 Objectives	6
	1.4 Scope and limitations	7
	1.5 Mapping to course modules and the lab experiment	7
2.	**LITERATURE SURVEY**	8
	2.1 Overview of the existing protocol or scheme	8
	2.2 Review of related work (table: author and year, technique, metrics, merits, gaps)	9
	2.3 Research gap and justification of novelty	9
3.	**PROPOSED SYSTEM**	11
	3.1 System architecture or topology (diagram)	11
	3.2 Baseline mechanism description	12
	3.3 Proposed mechanism: algorithm, pseudocode, flowchart	13
	3.4 Novelty statement (what exactly differs from the baseline)	15
	3.5 Assumptions and design decisions	16
4.	**IMPLEMENTATION**	17
	4.1 Tools, platform, versions and hardware	17
	4.2 Simulation or testbed setup (topology, configuration, parameter table)	17
	4.3 Code structure and explanation of key modules	18
	4.4 Challenges faced and how they were solved	19

Sl.No	Contents	Page No.
5.	**EXPERIMENTAL SETUP AND PERFORMANCE EVALUATION**	20
	5.1 Performance metrics and why they were chosen	20
	5.2 Scenarios and parameter values	20
	5.3 Results: tables and graphs, baseline vs proposed (mean of repeated runs)	20
	5.4 Analysis and discussion (reasons, trends, trade-offs)	22
	5.5 Critical evaluation: strengths, weaknesses, conditions where the proposed scheme fails	23
6.	**CONCLUSION AND FUTURE WORK**	25
	6.1 Summary of work and key findings (quantified)	25
	6.2 Limitations	25
	6.3 Future enhancements	25
7.	**REFERENCES**	26
	References (IEEE style)	26
	Appendix A: Important source code or configuration listings	27
	Appendix B: Additional screenshots, packet captures, raw result tables	30

1. INTRODUCTION

1.1 Background and motivation

The transport layer of the Internet offers two main services. The Transmission Control Protocol (TCP) provides a reliable, ordered byte stream, while the User Datagram Protocol (UDP) provides only a thin, connectionless datagram service with an 8-byte header and an optional checksum [1], [2]. UDP does not acknowledge, retransmit, reorder or de-duplicate anything, so datagrams may be lost, corrupted, duplicated or delivered out of order.

Many applications nevertheless need reliability without the constraints of kernel-resident TCP. Real systems therefore build reliability above UDP at the application level: Reliable UDP for signalling [7], the UDT protocol for high-speed bulk transfer [8] and, most visibly, QUIC, which now carries a large share of web traffic [9]. Building a small reliable protocol on UDP is therefore both a practical engineering task and the clearest way to understand the mechanisms that TCP hides: sequence numbers, acknowledgements, timers, windows and retransmission policies [10].

This project implements such a protocol and, following the evaluation practice required by the course, compares a baseline mechanism with a proposed mechanism under controlled and repeatable network impairments.

1.2 Problem statement

Given an unreliable datagram service that may lose, corrupt, duplicate or reorder packets, design and implement an application-layer protocol that transfers a file completely, correctly and in order, and quantify how different retransmission policies behave as network quality degrades.

1.3 Objectives

- Design a packet format with a sequence number, length and CRC-32 checksum, together with session set-up and teardown, on top of UDP sockets.
- Implement three ARQ schemes in one code base: Stop-and-Wait (baseline), Go-Back-N and Selective Repeat (proposed).
- Build a channel emulator that injects packet loss, bit corruption, delay and reordering in both directions.

- Verify the correctness of every transfer by comparing the SHA-256 hash of the sent and received files.
- Measure throughput, transfer time and retransmissions over repeated runs and report mean and standard deviation.
- Analyse the trade-offs and identify the conditions under which the proposed scheme brings no benefit or fails.

1.4 Scope and limitations

The system transfers one file between one sender and one receiver per session. Experiments run on the loopback interface with impairments injected by a software emulator, so no real queueing delay or congestion is present. Loss rates up to 20 percent, a fixed retransmission timeout and a 100 KB test file were used. Congestion control, adaptive timeout estimation, sequence-number wrap-around, encryption and multi-file or multi-connection operation are outside the scope of this work (see Section 6.2).

1.5 Mapping to course modules and the lab experiment

The project applies the following topics of the Computer Networks course and extends the socket-programming and sliding-window experiments of the laboratory (module numbers to be matched with the course plan).

Table 1.1: Mapping of the project to course topics

Course topic	How it appears in the project	Where
Transport layer services; UDP and TCP	UDP is the unreliable substrate; reliability is added above it	Chapters 2, 3
Principles of reliable data transfer	Stop-and-Wait, Go-Back-N and Selective Repeat implemented and compared	Sections 3.2, 3.3
Error detection	CRC-32 over header and payload; corrupted packets are discarded	Sections 3.1, 4.3
Connection management	SYN/SYNACK set-up and FIN/FINACK teardown	Section 3.1
Flow control and sliding window	Window size W limits packets in flight; window sweep experiment	Sections 3.3, 5.3
Socket programming (UDP)	Python socket API: sendto and recvfrom with timeouts	Chapter 4
Network performance evaluation	Throughput, delay and retransmissions over repeated runs	Chapter 5

2. LITERATURE SURVEY

2.1 Overview of the existing protocol or scheme

UDP. UDP [1] adds only port numbers, a length and a checksum to IP. It is fast and simple but provides no reliability, ordering or flow control. **TCP.** TCP [2], [3] adds connection management, byte-sequence numbers, cumulative acknowledgements, a sliding window and a retransmission timer whose value is estimated from measured round-trip times [6]. Congestion control was added later [4]. The selective acknowledgment option [5] lets a TCP receiver report non-contiguous blocks so that only missing segments are resent, which is the practical form of the Selective Repeat idea.

Classic ARQ schemes. Textbooks [10], [11] describe three automatic repeat request schemes. In *Stop-and-Wait* the sender transmits one packet and waits for its acknowledgement; it is simple but its throughput is limited to one packet per round-trip time. In *Go-Back-N* the sender keeps up to N packets in flight, the receiver accepts only the next in-order packet and acknowledges cumulatively, and a timeout makes the sender resend the whole window. In *Selective Repeat* the receiver buffers out-of-order packets and acknowledges each one, and the sender keeps a timer for every packet and resends only the ones that time out.

In this project Stop-and-Wait is the **baseline**, Go-Back-N is a second reference, and Selective Repeat is the **proposed mechanism**.

2.2 Review of related work (table: author and year, technique, metrics, merits, gaps)

Table 2.1: Literature survey

S.No	Authors, year	Technique	Metrics and tool	Merits	Gaps
1	J. Postel, 1980 [1]	UDP: connectionless datagrams with ports and checksum	Protocol specification (RFC 768)	Minimal overhead and delay; easy to build upon	No reliability, ordering or duplicate protection
2	J. Postel, 1981; W. Eddy, 2022 [2], [3]	TCP: sequence numbers, cumulative ACKs, sliding window, retransmission timer	Specification; widely deployed	Reliable, ordered delivery for all applications	Kernel-resident and not customisable; head-of-line blocking
3	V. Jacobson, 1988 [4]	Slow start, congestion avoidance, RTT-based timer	Throughput and stability; measurements on the Internet	Prevents congestion collapse	Treats every loss as congestion; not studied here for UDP
4	M. Mathis et al., 1996 [5]	TCP selective acknowledgment (SACK) blocks	Specification (RFC 2018)	Recovers several losses per window without resending good data	Needs option negotiation and extra header space
5	T. Bova and T. Krivoruchka, 1999 [7]	Reliable UDP: in-order delivery over UDP up to a maximum number of retransmissions	Internet-Draft (expired)	Simple; suited to telephony signalling	Not aimed at bulk transfer; no performance study in the draft
6	Y. Gu and R. L. Grossman, 2007 [8]	UDT: UDP-based transfer with its own reliability and congestion control	Throughput and fairness; high-speed wide-area testbeds	High utilisation on high bandwidth-delay paths	Complex; designed for long fat networks, not for teaching ARQ
7	J. Iyengar and M. Thomson, 2021 [9]	QUIC: multiplexed, encrypted streams over UDP	Specification (RFC 9000)	No cross-stream head-of-line blocking; fast handshakes	Large specification with integrated security; heavy for study
8	J. F. Kurose and K. W. Ross, 2021 [10]	Textbook models of Stop-and-Wait, Go-Back-N and Selective Repeat	Analytical and simulator-style description	Clear definition of each scheme and its trade-offs	No running code over real sockets, no repeated-run measurements

2.3 Research gap and justification of novelty

Production protocols such as TCP, QUIC and UDT are large and mix many concerns (security, congestion control, multiplexing), which makes it hard to see what a single retransmission policy contributes. Textbook treatments describe the ARQ schemes abstractly [10], [11], and student implementations commonly show one scheme on an ideal network or report single runs.

The gap addressed here is a compact, fully runnable implementation of the three ARQ schemes over real UDP sockets that (i) use the same packet format, checksum, timer and test file so that the retransmission policy is the only variable, (ii) include an emulator for loss, corruption, delay and reordering, and (iii) are evaluated with repeated runs, mean and standard deviation, and an explicit analysis of where the proposed scheme does *not* help. The contribution is an

engineering and evaluation contribution; Selective Repeat itself is a standard algorithm [5], [10].

3. PROPOSED SYSTEM

3.1 System architecture or topology (diagram)

The system consists of a sender process and a receiver process that communicate through UDP sockets. A channel emulator wraps each socket and applies loss, corruption, delay and reordering to every packet it sends, so data packets and acknowledgements are both subject to impairment (Figure 3.1).

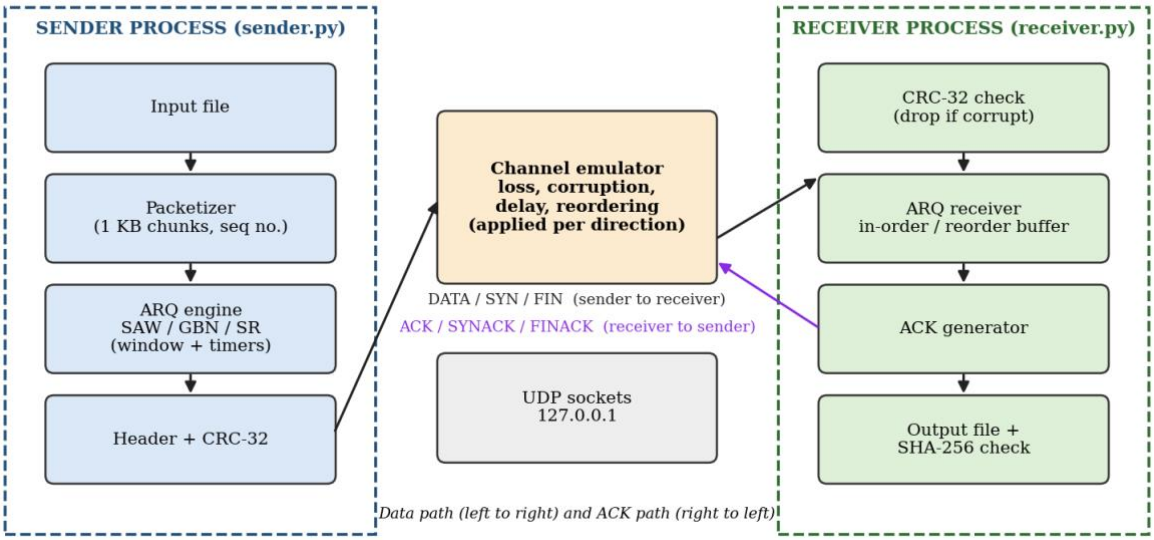


Figure 3.1: System architecture

Every packet carries an 11-byte header followed by up to 1024 bytes of payload (Figure 3.2). The CRC-32 is computed over the header (with the checksum field set to zero) and the payload; a packet whose checksum does not match is discarded exactly as if it had been lost, and the retransmission timer repairs it.

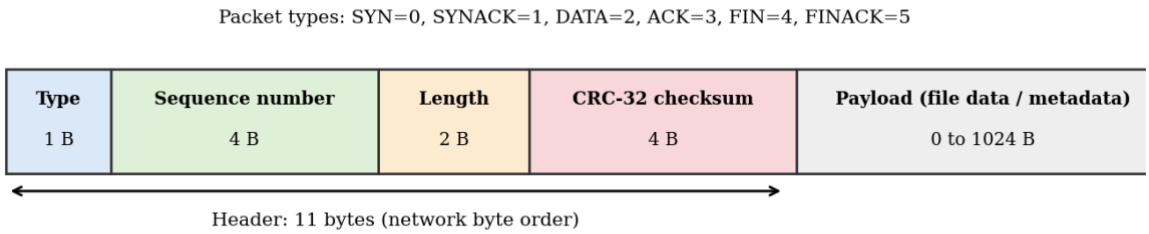


Figure 3.2: Packet format

Table 3.1: Packet types

Type	Direction	Meaning and use of the sequence field
SYN	Sender to receiver	Opens a session; payload is JSON with file name, size, packet count, ARQ mode and window
SYNACK	Receiver to sender	Accepts the session; the SYN is repeated until this arrives
DATA	Sender to receiver	One 1024-byte chunk of the file; sequence number is the chunk index
ACK	Receiver to sender	SAW and GBN: sequence = next expected chunk (cumulative). SR: sequence = the chunk acknowledged
FIN	Sender to receiver	All chunks are acknowledged; request to close
FINACK	Receiver to sender	Close confirmed; the receiver lingers briefly to answer a repeated FIN

A transfer therefore has three phases: session set-up (SYN, SYNACK), data transfer (DATA and ACK packets under the chosen ARQ scheme) and teardown (FIN, FINACK). At the end both sides print the SHA-256 hash of the file, and equal hashes prove that the received file is identical to the original.

3.2 Baseline mechanism description

The baseline is **Stop-and-Wait**. The sender transmits packet *n*, starts a timer and waits. If the acknowledgement arrives, the sender moves to packet *n+1*; if the timer expires first, packet *n* is sent again. The receiver accepts packets in order and acknowledges the next expected sequence number. Because only one packet is outstanding, the link is idle for most of each round trip, and the best possible throughput is

$$Throughput \leq L / (RTT + t_tx)$$

where L is the payload size. With L = 1024 bytes and a round-trip time of about 10.8 ms in the testbed (two 5 ms emulated one-way delays plus processing), this bound is about 95 KB/s, which agrees with the measured 95.0 KB/s at zero loss (Table 5.2). Every loss adds an idle wait of one full timeout.

Go-Back-N is used as a second reference. It allows W packets in flight and uses one timer for the oldest unacknowledged packet. The receiver discards out-of-order packets and repeats the last cumulative acknowledgement; on a timeout the sender resends all packets from the oldest unacknowledged one. Table 3.2 compares the three schemes.

Table 3.2: Comparison of the implemented ARQ schemes

Property	Stop-and-Wait (baseline)	Go-Back-N	Selective Repeat (proposed)
Packets in flight	1	Up to W	Up to W
Acknowledgement	Cumulative	Cumulative	Individual, per packet
Sender timers	One	One (oldest packet)	One per packet
Receiver buffering	None	None (discards out-of-order)	Buffers out-of-order packets (up to W)
Action on loss	Resend the packet	Resend the whole window	Resend only the lost packet
Complexity	Lowest	Medium	Highest

3.3 Proposed mechanism: algorithm, pseudocode, flowchart

The proposed mechanism is **Selective Repeat** with a window of W packets, per-packet timers and individual acknowledgements. The sender window covers sequence numbers [base, base+W). Each packet that is sent starts its own timer; when an acknowledgement for packet s arrives, s is marked as acknowledged and base advances past every contiguous acknowledged packet. If the timer of a packet expires, only that packet is resent. The receiver accepts any packet in [rcv_base, rcv_base+W), stores it, acknowledges exactly that packet, and delivers data to the file in order as soon as the gap is filled. A packet below rcv_base is a duplicate (usually caused by a lost ACK) and is re-acknowledged but not delivered again.

Sender pseudocode

```
base <- 0 ; next <- 0                          # N = number of packets, W = window
send SYN ; wait for SYNACK (repeat SYN on timeout)
while base < N:
    while next < min(base + W, N):
        send DATA(next) ; timer[next] <- now ; next <- next + 1
    wait for an ACK until (earliest timer + T)
    if ACK(s) is valid and base <= s < next and not acked[s]:
        acked[s] <- true ; delete timer[s]
        while acked[base]: base <- base + 1       # slide the window
    for each s with now - timer[s] >= T:
        send DATA(s) ; timer[s] <- now            # selective retransmission
send FIN ; wait for FINACK (repeat FIN on timeout)
```

Receiver pseudocode

```
rcv_base <- 0 ; buffer <- {}
on packet DATA(s) with valid CRC-32:
    if s < rcv_base:                              # duplicate
        send ACK(s)
    else if s < rcv_base + W:
        if s not in buffer: buffer[s] <- payload
        send ACK(s)                               # individual acknowledgement
        while rcv_base in buffer:
            write buffer[rcv_base] to file ; rcv_base <- rcv_base + 1
    else: ignore                                  # outside the window
on packet FIN when all N packets are delivered: send FINACK ; linger ; close file
```

Figure 3.3 shows the flowcharts of the sender and receiver, and Figure 3.4 contrasts the behaviour of Go-Back-N and Selective Repeat when a single packet is lost.

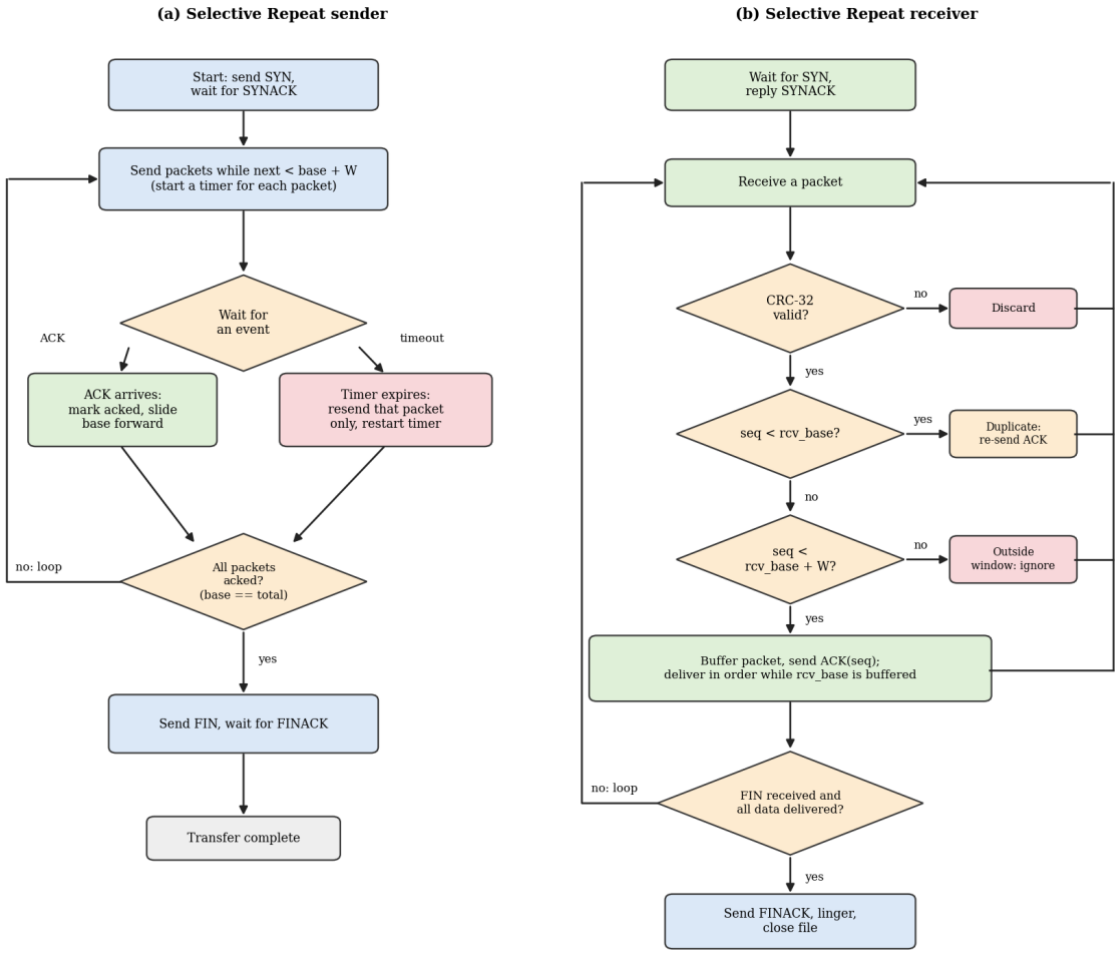


Figure 3.3: Flowcharts of the Selective Repeat sender and receiver

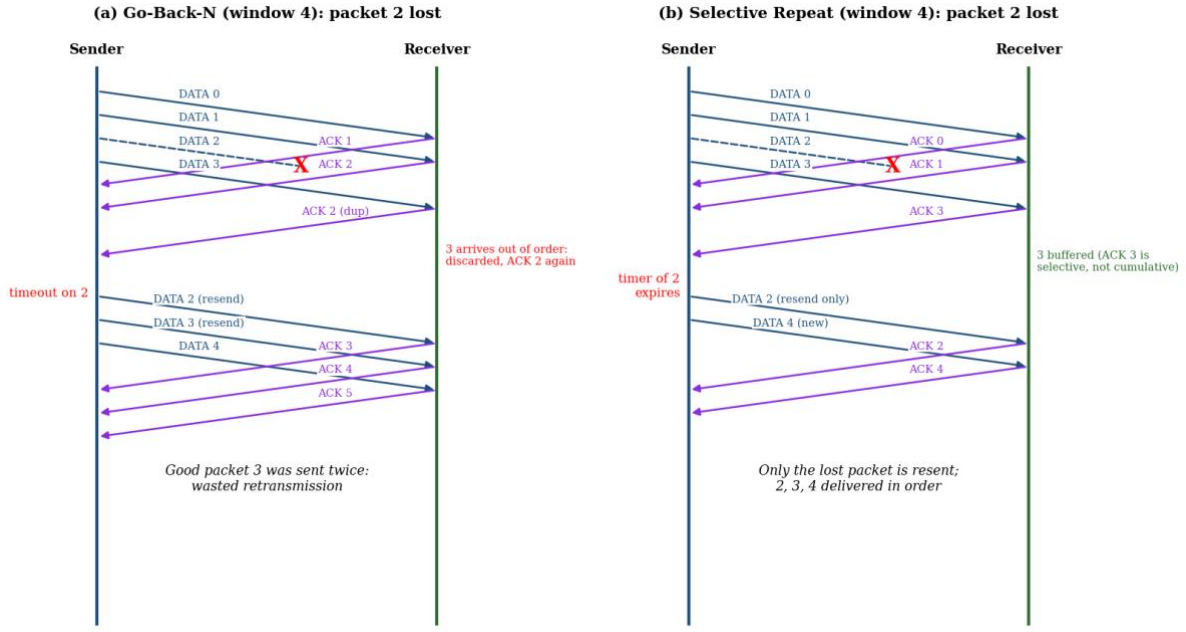


Figure 3.4: Loss of one packet under Go-Back-N and Selective Repeat (window 4)

3.4 Novelty statement (what exactly differs from the baseline)

Relative to the Stop-and-Wait baseline the proposed mechanism differs in four algorithmic respects: it pipelines up to W packets instead of one; it keeps one timer per packet instead of one timer in total; it uses individual acknowledgements so that the sender knows exactly which packets are missing; and its receiver buffers out-of-order packets so that a loss never causes good data to be discarded or resent. Relative to Go-Back-N the difference is that only the lost packet is resent.

The novelty claimed for this work is not a new ARQ algorithm, because Selective Repeat is standard [5], [10]. It lies in the integration and the evaluation method: (1) a single implementation in which the ARQ policy is the only variable, with the same header, CRC-32, timeout, window size and file for all schemes; (2) a built-in emulator that applies loss, corruption, delay and reordering to data and acknowledgements; (3) a paired experimental design in which every scheme sees the same random seeds; and (4) an explicit search for conditions where the proposed scheme brings no benefit (Section 5.5).

3.5 Assumptions and design decisions

Table 3.3: Assumptions and design decisions

Decision	Value	Reason
Payload size	1024 bytes	Well below the typical 1500-byte Ethernet MTU, so no IP fragmentation
Header	11 bytes, CRC-32	Compact; zlib.crc32 detects all single-bit and burst errors in a packet with high probability
Sequence numbers	32-bit, no wrap-around	A 100 KB test file needs about 100 numbers; wrap-around is left as future work
Retransmission timeout	Fixed 60 ms	About six times the 10.8 ms emulated round trip; simple and identical for all schemes
Window size	8 (1 for Stop-and-Wait)	Gives a theoretical bound near 758 KB/s without overflowing the receiver buffer
Corrupted packets	Treated as lost	Avoids negative acknowledgements; the timer repairs the loss
End of session	FIN/FINACK with receiver linger	A lost FINACK or final ACK can still be repaired
Network	Loopback with emulator	Repeatable results; loss and delay are controlled exactly

4. IMPLEMENTATION

4.1 Tools, platform, versions and hardware

Table 4.1: Tools and platform

Item	Details
Programming language	Python 3.12 (standard library only for the protocol)
Modules used	socket, struct, zlib (CRC-32), threading, heapq, hashlib (SHA-256), json, argparse, statistics, csv
Plotting	matplotlib 3.10 (graphs only; not needed by the protocol)
Editor	Visual Studio Code
Development and functional testing	Apple MacBook Air, macOS, Python 3.12
Measurement platform for Chapter 5	Ubuntu 24.04.4 LTS, Python 3.12.3, Intel Xeon 2.80 GHz (1 vCPU), 3 GB RAM, loopback interface 127.0.0.1
Network	Software emulation of loss, corruption, delay and reordering (no physical link)

4.2 Simulation or testbed setup (topology, configuration, parameter table)

The sender and receiver run as two threads of one process (in the benchmark) or as two terminals (in the demonstration), each with its own UDP socket on 127.0.0.1. Each endpoint sends through its own Channel object, so the forward path (data) and the reverse path (ACKs) are impaired independently. Each run creates fresh sockets, transfers the file, checks the SHA-256 hash and discards the received copy.

Table 4.2: Experiment parameters

Parameter	Value
Test file	100 KB of random bytes (100 packets of 1024 bytes)
One-way emulated delay	5 ms (round trip about 10.8 ms)
Retransmission timeout	60 ms (fixed)
Window size	1 (Stop-and-Wait), 8 (Go-Back-N and Selective Repeat); 1 to 32 in the window sweep
Packet loss (each direction)	0, 5, 10, 20 percent
Jitter (reordering scenario S4)	0 to 10 ms of random extra delay per packet
Repeated runs	10 per scenario (5 per point in the window sweep)
Random seeds	Seeds 100, 113, ..., 217; the same seeds for every scheme (paired design)
Statistics reported	Mean and sample standard deviation (n - 1)

4.3 Code structure and explanation of key modules

Table 4.3: Code structure

File	Responsibility
rdt_common.py	Packet format (make_packet, parse_packet with CRC-32) and the Channel class: random loss, single-bit corruption, delay and reordering using a time-ordered heap and a background delivery thread
sender.py	send_file(): SYN handshake, then the Stop-and-Wait / Go-Back-N loop or the Selective Repeat loop, then FIN teardown; command-line interface and statistics
receiver.py	receive_file(): handshake, in-order delivery for SAW/GBN, reorder buffer for SR, duplicate and corruption counters, linger after FIN, SHA-256 of the output
test_transfer.py	Correctness test of the three schemes, with and without a bad network, verified by SHA-256
benchmark_stats.py	Repeated-run experiments, raw_runs.csv, results_stats.csv and the graphs with error bars

Packet handling. make_packet() builds the header with struct.pack, computes the CRC-32 over header and payload and returns the final bytes. parse_packet() recomputes the checksum and returns None for any packet that is too short, has an inconsistent length, an unknown type or a bad checksum.

Channel emulator. For every packet the channel draws random numbers to decide on loss and corruption, and schedules delivery at the current time plus the delay and jitter. Delayed packets are kept in a heap and released by a background thread, which preserves ordering when there is no jitter and produces reordering when there is.

Sender. In Stop-and-Wait and Go-Back-N mode the sender keeps base, next and a single timer; an acknowledgement a with base < a <= next slides the window. In Selective Repeat mode it keeps an acknowledged flag and a last-send time for each packet and waits only until the nearest timer expires. **Receiver.** In SR mode packets in [expected, expected + W) are stored in a dictionary and written to the file as soon as the next expected sequence number is present. The key parts of the code are listed in Appendix A.

4.4 Challenges faced and how they were solved

Table 4.4: Challenges and solutions

Challenge	Cause	Solution
The last ACK or the FINACK can be lost, leaving the sender waiting	The receiver exits as soon as it has the data	FIN/FINACK exchange and a receiver linger period that keeps answering repeated FINs and duplicate packets
Go-Back-N looked bad even with 0 percent loss in early tests	2 ms of random jitter reordered packets that were sent microseconds apart, and Go-Back-N discards out-of-order packets	Main experiments use an in-order network; reordering is evaluated separately as scenario S4
Waiting with a zero timeout	settimeout(0) makes a Python socket non-blocking and raises an error	recv() clamps the timeout to at least 1 ms
Selective Repeat needs the nearest of many timers	One timer per packet	The loop computes the earliest expiry and waits only that long, then scans the unacknowledged packets
Unrealistic reordering from a simple sleep	A sleep per packet cannot keep order or create it consistently	A heap ordered by due time and one delivery thread
Operating-system differences	Some systems raise ConnectionResetError on UDP when the peer has closed	recv() treats it like a timeout

5. EXPERIMENTAL SETUP AND PERFORMANCE EVALUATION

5.1 Performance metrics and why they were chosen

- **Throughput (KB/s)** = file size divided by transfer time. It shows how well each scheme uses the link and is the metric a user notices.
- **Transfer time (s)**, measured from the end of the handshake until the last packet is acknowledged. It is the inverse view of throughput and is easier to interpret in absolute terms.
- **Retransmissions** = data packets sent more than once. It measures wasted effort and separates Go-Back-N from Selective Repeat, which time and throughput alone do not.
- **Correctness** = SHA-256 of the received file equals that of the original. All 300 or more measured transfers passed this check; a mismatch would have stopped the benchmark.

Each value is reported as mean ± sample standard deviation over repeated runs. Improvement is computed as (proposed - baseline) / baseline for throughput, and as (baseline - proposed) / baseline for time and retransmissions, so that a positive percentage always means an improvement.

5.2 Scenarios and parameter values

Table 5.1: Scenarios

Scenario	Network condition	Schemes compared
S0 (reference)	0 percent loss, 5 ms delay	SAW, GBN, SR
S1 (low loss)	5 percent loss each direction	SAW, GBN, SR
S2 (medium loss)	10 percent loss each direction	SAW, GBN, SR
S3 (high loss)	20 percent loss each direction	SAW, GBN, SR
S4 (reordering)	5 percent loss and 0 to 10 ms random jitter	GBN, SR
Window sweep	10 percent loss, window 1, 2, 4, 8, 16, 32	GBN, SR

The scenarios are labelled by network impairment rather than by traffic load, because in a single-flow transfer the loss rate is the quantity that varies the difficulty.

5.3 Results: tables and graphs, baseline vs proposed (mean of repeated runs)

Table 5.2: Reference scenario S0 (no loss), 10 runs

Scheme	Throughput (KB/s), mean ± SD	Transfer time (s), mean ± SD
Stop-and-Wait	95.0 ± 0.6	1.053 ± 0.006
Go-Back-N	726.0 ± 7.6	0.138 ± 0.001
Selective Repeat	725.2 ± 6.1	0.138 ± 0.001

Table 5.3: Baseline (Stop-and-Wait) vs proposed (Selective Repeat), 10 runs per cell

Scenario	Metric	Baseline (mean ± SD)	Proposed (mean ± SD)	Improvement (%)
S1 (low loss, 5%)	Throughput (KB/s)	63.0 ± 9.1	213.1 ± 67.6	+238.3
	Transfer time (s)	1.61 ± 0.20	0.50 ± 0.11	+69.0
	Retransmissions	9.3 ± 3.4	9.3 ± 3.4	+0.0
S2 (medium loss, 10%)	Throughput (KB/s)	42.5 ± 4.8	129.7 ± 23.0	+205.2
	Transfer time (s)	2.38 ± 0.26	0.79 ± 0.12	+66.8
	Retransmissions	22.0 ± 4.4	22.0 ± 4.4	+0.0
S3 (high loss, 20%)	Throughput (KB/s)	23.7 ± 3.0	72.4 ± 11.3	+205.4
	Transfer time (s)	4.29 ± 0.62	1.41 ± 0.20	+67.1
	Retransmissions	53.5 ± 10.3	53.5 ± 10.3	+0.0

The retransmission counts of Stop-and-Wait and Selective Repeat are identical because both saw the same loss pattern (same seeds); Selective Repeat does not resend fewer packets than Stop-and-Wait, it finishes sooner because it overlaps the recovery with new transmissions.

Table 5.4: Go-Back-N vs Selective Repeat, 10 runs per cell

Scenario	Metric	Go-Back-N (mean ± SD)	Selective Repeat (mean ± SD)	Improvement (%)
S1 (low loss, 5%)	Throughput (KB/s)	230.2 ± 94.7	213.1 ± 67.6	-7.4
	Transfer time (s)	0.49 ± 0.16	0.50 ± 0.11	-2.7
	Retransmissions	39.3 ± 19.5	9.3 ± 3.4	+76.3
S2 (medium loss, 10%)	Throughput (KB/s)	117.0 ± 34.7	129.7 ± 23.0	+10.9
	Transfer time (s)	0.92 ± 0.25	0.79 ± 0.12	+13.9
	Retransmissions	90.6 ± 32.7	22.0 ± 4.4	+75.7
S3 (high loss, 20%)	Throughput (KB/s)	57.6 ± 9.0	72.4 ± 11.3	+25.8
	Transfer time (s)	1.78 ± 0.31	1.41 ± 0.20	+20.8
	Retransmissions	187.9 ± 35.6	53.5 ± 10.3	+71.5

Table 5.5: Reordering scenario S4, 10 runs

Scenario	Metric	Go-Back-N (mean ± SD)	Selective Repeat (mean ± SD)	Improvement (%)
S4 (5% loss + reordering)	Throughput (KB/s)	19.9 ± 1.5	166.2 ± 30.7	+733.0
	Transfer time (s)	5.04 ± 0.39	0.62 ± 0.09	+87.8
	Retransmissions	474.6 ± 37.9	9.3 ± 3.4	+98.0

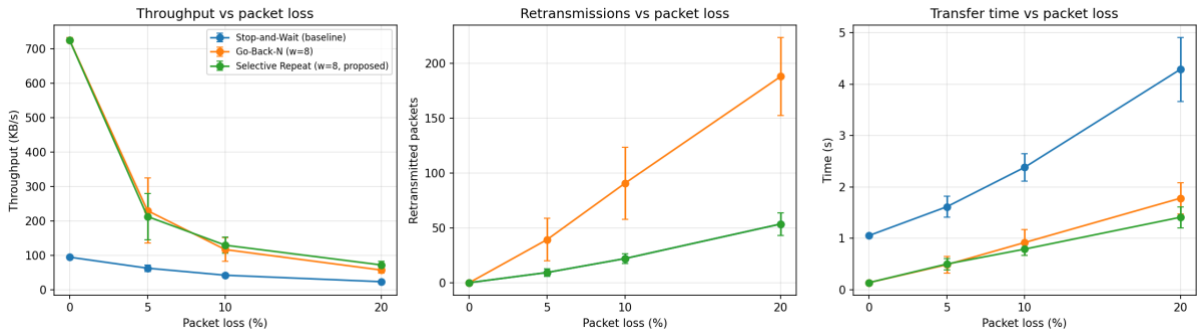


Figure 5.1: Throughput, retransmissions and transfer time against packet loss (mean ± SD, 10 runs)

Table 5.6: Effect of window size at 10 percent loss, 5 runs per cell

Window	GBN time (s)	SR time (s)	GBN retransmissions	SR retransmissions
1	2.52 ± 0.19	2.51 ± 0.20	24.2 ± 3.3	24.2 ± 3.3
2	1.59 ± 0.31	1.75 ± 0.24	32.8 ± 9.8	24.2 ± 3.3
4	1.16 ± 0.44	1.30 ± 0.22	55.0 ± 27.4	24.2 ± 3.3
8	0.97 ± 0.30	0.81 ± 0.16	97.8 ± 37.2	24.2 ± 3.3
16	0.89 ± 0.33	0.55 ± 0.11	185.6 ± 66.3	24.2 ± 3.3
32	0.68 ± 0.10	0.38 ± 0.07	269.0 ± 68.7	24.2 ± 3.3

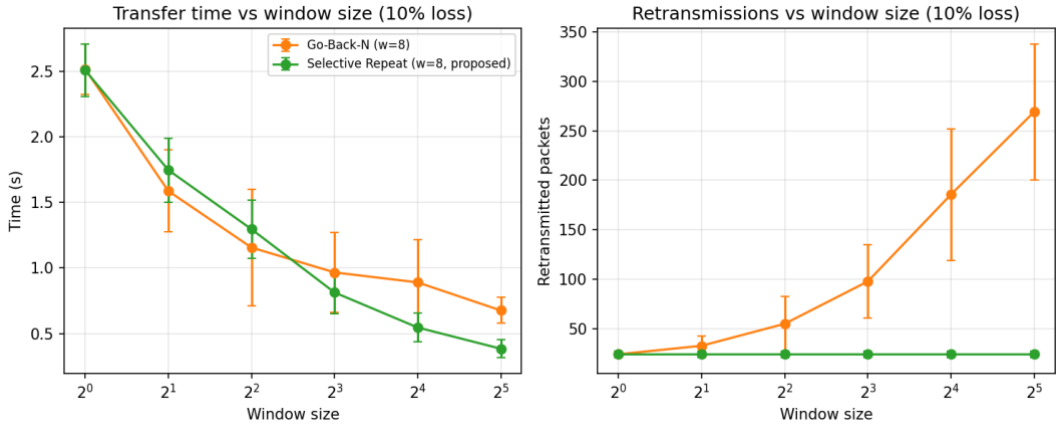


Figure 5.2: Effect of window size on transfer time and retransmissions (10 percent loss)

5.4 Analysis and discussion (reasons, trends, trade-offs)

Windowing removes the round-trip bottleneck. At zero loss Stop-and-Wait reaches 95.0 KB/s, matching the bound L/RTT of Section 3.2, while Go-Back-N and Selective Repeat with W = 8 reach about 726 KB/s, close to the bound W·L/RTT of about 758 KB/s (Table 5.2). Both windowed schemes behave identically when nothing is lost, as expected.

Loss hurts Stop-and-Wait most. Every loss idles the link for a full 60 ms timeout, so its throughput falls from 95 to 63, 43 and 24 KB/s at 5, 10 and 20 percent loss. Selective Repeat

improves on it by 238, 205 and 205 percent in throughput and by about 67 to 69 percent in time (Table 5.3), because loss recovery overlaps with the transmission of other packets.

Go-Back-N wastes retransmissions. Selective Repeat needs 76.3, 75.7 and 71.5 percent fewer retransmissions than Go-Back-N at 5, 10 and 20 percent loss (Table 5.4), roughly one quarter of the packets Go-Back-N resends, because Go-Back-N resends good packets that follow a lost one. The waste grows with the window: Go-Back-N retransmissions rise from 24 to 269 as the window grows from 1 to 32, while Selective Repeat stays at 24.2 (Table 5.6), which is the number of lost transmissions itself.

Time and throughput differences are smaller than retransmission differences. At 10 and 20 percent loss Selective Repeat is 13.9 and 20.8 percent faster than Go-Back-N, and in the window sweep its time falls from 2.51 s at W = 1 to 0.38 s at W = 32 (85 percent), against 2.52 s to 0.68 s (73 percent) for Go-Back-N. At 5 percent loss the two are not distinguishable: the means differ by 2.7 percent in time and 7.4 percent in throughput in favour of Go-Back-N, well inside one standard deviation. A cumulative ACK in Go-Back-N also covers earlier lost ACKs, whereas in Selective Repeat every lost ACK causes an unnecessary retransmission; at low loss this offsets the saving.

Reordering is where the schemes differ most. With 5 percent loss and random jitter, Go-Back-N delivers only 19.9 ± 1.5 KB/s and resends 475 packets, because every out-of-order arrival is discarded and triggers a timeout, whereas Selective Repeat buffers those packets and reaches 166.2 ± 30.7 KB/s with 9.3 retransmissions (Table 5.5), an improvement of 98 percent in retransmissions and 88 percent in time.

5.5 Critical evaluation: strengths, weaknesses, conditions where the proposed scheme fails

Strengths

- Every one of the more than 300 measured transfers was byte-identical to the original, so the protocol is correct under loss, corruption and reordering.
- Retransmissions are minimal and independent of the window size, and the scheme tolerates reordering.
- The transfer time keeps falling as the window grows, up to the largest window tested (32).

Weaknesses and conditions where it does not help or may fail

- **No advantage at low loss.** At 5 percent loss Selective Repeat was not better than Go-Back-N (-7.4 percent throughput, -2.7 percent time, within noise), while it costs more memory and code.
- **Fixed timeout.** The 60 ms timeout suits a round trip of about 11 ms. On a path with a much longer or variable round trip, a fixed timeout would cause spurious retransmissions or long idle waits. This was not measured and is a limitation of the design; TCP uses an adaptive timer [6].
- **Lost acknowledgements cost more than in Go-Back-N** because there is no cumulative acknowledgement to cover them.
- **High variance.** With about 100 packets per transfer only a few losses occur, so standard deviations are large (for example Go-Back-N at 5 percent: 230 ± 95 KB/s). More runs or a larger file would tighten the estimates.
- **Limits of the testbed.** The emulation runs on loopback on a single virtual CPU, so thread scheduling adds noise and there is no real queueing or congestion. Loss above 20 percent and other file sizes were not tested.
- **No congestion control.** A real deployment would need it; otherwise the sender could overload a shared link.

6. CONCLUSION AND FUTURE WORK

6.1 Summary of work and key findings (quantified)

A reliable data transfer protocol was designed and implemented on top of UDP, with session set-up and teardown, CRC-32 error detection, sequence numbers, acknowledgements, timers and a sliding window, in three ARQ variants sharing one packet format. A channel emulator allowed repeatable experiments, and the SHA-256 comparison confirmed that all transfers were correct. The main findings are:

- Windowing raises throughput from 95 KB/s to about 726 KB/s on a clean network (7.6 times).
- Compared with the Stop-and-Wait baseline, Selective Repeat gave 238, 205 and 205 percent higher throughput and 69, 67 and 67 percent lower transfer time at 5, 10 and 20 percent loss.
- Compared with Go-Back-N, Selective Repeat needed 71 to 76 percent fewer retransmissions at all loss levels and was 14 and 21 percent faster at 10 and 20 percent loss, but not better at 5 percent.
- Under reordering, Selective Repeat reached 166 KB/s against 20 KB/s for Go-Back-N.

6.2 Limitations

The timeout is fixed, there is no congestion or flow control beyond the window, sequence numbers do not wrap, only one file and one connection are supported, and the evaluation used an emulated loopback network with a 100 KB file and loss up to 20 percent.

6.3 Future enhancements

- Adaptive retransmission timeout from measured round-trip times, as in TCP [6].
- Congestion control (slow start and additive-increase multiplicative-decrease) [4].
- Cumulative plus selective acknowledgements, in the style of TCP SACK [5], to remove the penalty of lost ACKs.
- Sequence-number wrap-around, multiple files and concurrent connections.
- Evaluation on a real network or an emulator such as Mininet, with larger files, more runs and statistical significance tests.
- Authentication and encryption of packets.

7. REFERENCES

References (IEEE style)

[1] J. Postel, "User Datagram Protocol," IETF, RFC 768, Aug. 1980.

[2] J. Postel, "Transmission Control Protocol," IETF, RFC 793, Sep. 1981.

[3] W. Eddy, Ed., "Transmission Control Protocol (TCP)," IETF, RFC 9293, Aug. 2022.

[4] V. Jacobson, "Congestion avoidance and control," in *Proc. ACM SIGCOMM*, Stanford, CA, USA, Aug. 1988, pp. 314-329.

[5] M. Mathis, J. Mahdavi, S. Floyd, and A. Romanow, "TCP selective acknowledgment options," IETF, RFC 2018, Oct. 1996.

[6] V. Paxson, M. Allman, J. Chu, and M. Sargent, "Computing TCP's retransmission timer," IETF, RFC 6298, Jun. 2011.

[7] T. Bova and T. Krivoruchka, "Reliable UDP protocol," IETF, Internet-Draft draft-ietf-sigtran-reliable-udp-00, Feb. 1999 (expired).

[8] Y. Gu and R. L. Grossman, "UDT: UDP-based data transfer for high-speed wide area networks," *Computer Networks*, vol. 51, no. 7, pp. 1777-1799, May 2007.

[9] J. Iyengar and M. Thomson, Eds., "QUIC: A UDP-based multiplexed and secure transport," IETF, RFC 9000, May 2021.

[10] J. F. Kurose and K. W. Ross, *Computer Networking: A Top-Down Approach*, 8th ed. Harlow, U.K.: Pearson, 2021.

[11] D. Bertsekas and R. Gallager, *Data Networks*, 2nd ed. Englewood Cliffs, NJ, USA: Prentice-Hall, 1992.

[12] Python Software Foundation, "socket - Low-level networking interface," *Python 3.12 Documentation*. [Online]. Available: https://docs.python.org/3.12/library/socket.html

Appendix A: Important source code or configuration listings

A.1 Packet construction and validation (rdt_common.py)

```
def make_packet(ptype, seq, payload=b""):
    """Build a packet and fill in the CRC-32 checksum."""
    head = HEADER.pack(ptype, seq, len(payload), 0)
    crc = zlib.crc32(head + payload) & 0xFFFFFFFF
    return HEADER.pack(ptype, seq, len(payload), crc) + payload



def parse_packet(raw):
    """Return (type, seq, payload), or None if the packet is malformed/corrupt."""
    if len(raw) < HEADER.size:
        return None
    ptype, seq, length, crc = HEADER.unpack_from(raw)
    payload = raw[HEADER.size:]
    if ptype > FINACK or len(payload) != length:
        return None
    check = zlib.crc32(HEADER.pack(ptype, seq, length, 0) + payload) & 0xFFFFFFFF
    if check != crc:
        return None
    return ptype, seq, payload
```

A.2 Channel emulator: loss, corruption and delay (rdt_common.py)

```
    def send(self, data, addr):
        self.sent += 1
        if self.rng.random() < self.loss:
            self.dropped += 1
            return
        if self.rng.random() < self.corrupt:
            buf = bytearray(data)
            buf[self.rng.randrange(len(buf))] ^= 1 << self.rng.randrange(8)
            data = bytes(buf)
            self.corrupted += 1
        if self._thread is None:
            self. raw send(data, addr)
        else:
            due = time.monotonic() + self.delay + self.rng.uniform(0, self.jitter)
            with self._cv:
                heapq.heappush(self._heap, (due, next(self._ids), data, addr))
                self._cv.notify()
```

A.3 Sender: Go-Back-N / Stop-and-Wait loop and Selective Repeat loop (sender.py)

```
    # ---- data transfer -------------------------------------------------------
    t0 = time.monotonic()
    last_progress = t0
    base = nxt = 0


    if mode in ("saw", "gbn"):
        timer = None
        while base < total:
            while nxt < min(base + window, total):
                send(nxt)
                if base == nxt:
                    timer = time.monotonic() + timeout
                nxt += 1
            read_ack.wait = max(0.0, (timer or 0) - time.monotonic())
            a = read_ack()                          # cumulative: a = next expected
            if a is not None and base < a <= nxt:
                base = a
                last_progress = time.monotonic()
                timer = time.monotonic() + timeout if base < nxt else None
            if timer is not None and time.monotonic() >= timer:
                st["timeouts"] += 1
                for s in range(base, nxt):       # go back N: resend everything
                    send(s, retrans=True)
                timer = time.monotonic() + timeout
            if time.monotonic() - last_progress > max_idle:
                raise TimeoutError("no progress - receiver unreachable?")
    else:  # Selective Repeat
        acked = [False] * total
        sent_at = {}                            # seq -> time of last send (unacked only)
        while base < total:
            while nxt < min(base + window, total):
                send(nxt)
```

```
                sent_at[nxt] = time.monotonic()
                nxt += 1
            nearest = min(sent_at.values()) + timeout if sent_at else time.monotonic() + timeout
            read_ack.wait = max(0.0, nearest - time.monotonic())
            s = read_ack()                      # individual ACK for packet s
            if s is not None and base <= s < nxt and not acked[s]:
                acked[s] = True
                sent_at.pop(s, None)
                last_progress = time.monotonic()
                while base < total and acked[base]:
                    base += 1
            now = time.monotonic()
            for seq, t in list(sent_at.items()):  # per-packet timers
                if now - t >= timeout:
                    st["timeouts"] += 1
                    send(seq, retrans=True)
                    sent_at[seq] = now
            if now - last_progress > max_idle:
                raise TimeoutError("no progress - receiver unreachable?")
```

A.4 Receiver: in-order delivery and Selective Repeat buffering (receiver.py)

```
        elif ptype == DATA:
            if seq >= meta["total"]:
                continue
            if meta["mode"] in ("saw", "gbn"):
                # Go-Back-N receiver: accept only the next expected packet
                if seq == expected:
                    out.write(payload)
                    expected += 1
                    st["delivered"] += 1
                    log(f"Delivered seq={seq}")
                elif seq < expected:
                    st["duplicates"] += 1
                    log(f"Duplicate packet seq={seq} discarded")
                else:
                    st["out_of_order"] += 1
                    log(f"Out-of-order seq={seq} discarded (expected {expected})")
                ack(expected)               # cumulative ACK
            else:
                # Selective Repeat receiver: buffer out-of-order packets
                w = meta["window"]
                if seq < expected:
                    st["duplicates"] += 1
                    log(f"Duplicate packet seq={seq} (re-ACKed)")
                    ack(seq)
                elif seq < expected + w:
                    if seq in buf:
                        st["duplicates"] += 1
                        log(f"Duplicate packet seq={seq} (re-ACKed)")
                    else:
                        buf[seq] = payload
                        if seq != expected:
                            st["out_of_order"] += 1
                            st["buffered"] += 1
                            log(f"Buffered out-of-order seq={seq} (expected {expected})")
                    ack(seq)                # individual ACK
                    while expected in buf:
                        out.write(buf.pop(expected))
                        st["delivered"] += 1
                        log(f"Delivered seq={expected}")
                        expected += 1
```

A.5 Configuration and commands

```
# correctness test (all three schemes, clean and bad network)
python3 test_transfer.py


# demonstration: terminal 1 (start first)
python3 receiver.py --port 9000 --out received.bin --verbose
# demonstration: terminal 2
python3 sender.py sample.bin --port 9000 --mode sr --window 8 --timeout 0.05 --loss 0.1 --verbose
shasum -a 256 sample.bin received.bin


# repeated-run evaluation used in Chapter 5
python3 benchmark_stats.py --size-kb 100 --repeat 10 --repeat-b 5


# options: --mode saw|gbn|sr  --window N  --timeout S  --loss P  --corrupt P  --delay S  --jitter S
--seed N
```

Appendix B: Additional screenshots, packet captures, raw result tables

B.1 Automated correctness test

All three schemes delivered an identical file on a clean network and with 15 percent loss, 5 percent corruption and jitter.

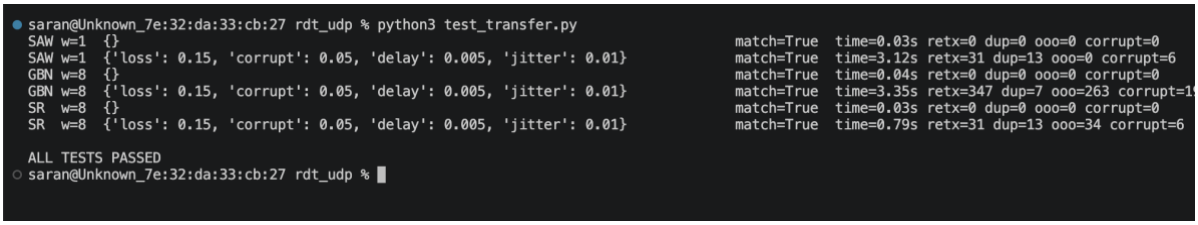


Figure B.1: Output of test_transfer.py (ALL TESTS PASSED)

B.2 Demonstration with matching hashes

Selective Repeat, window 8, 10 percent loss: 10 packets were dropped by the emulator and exactly 10 were retransmitted; the SHA-256 values of the sent and received files are identical.

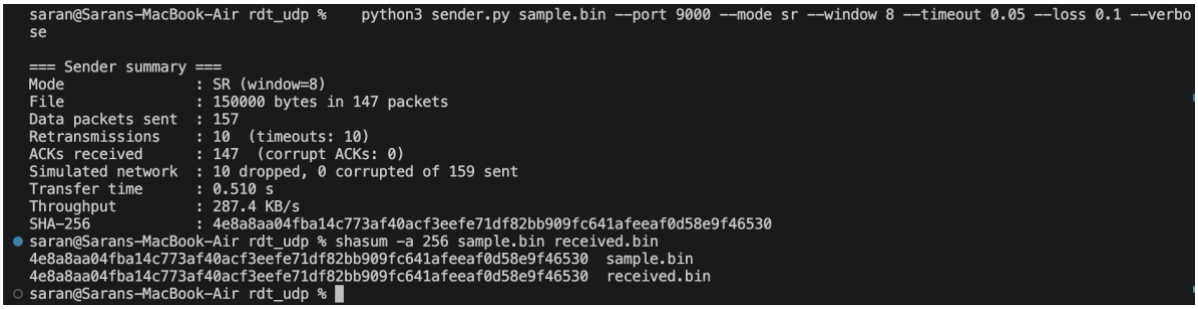


Figure B.2: Sender summary and hash comparison

B.3 Protocol log (8 KB file, Selective Repeat, window 4, 25 percent loss)

```
[sender] Sent SYN
[receiver] SYN received: tiny.bin (8192 bytes, 8 packets, mode=sr, window=4)
[sender] Received SYNACK - connection established
[sender] Sent packet seq=0
[sender] Sent packet seq=1
[sender] Sent packet seq=2
[sender] Sent packet seq=3
[receiver] Buffered out-of-order seq=2 (expected 0)
[receiver] Buffered out-of-order seq=3 (expected 0)
[sender] ACK received for 2
[sender] Timeout, retransmitting seq=0
[sender] Timeout, retransmitting seq=1
[sender] Timeout, retransmitting seq=3
[receiver] Delivered seq=0
[receiver] Duplicate packet seq=3 (re-ACKed)
[sender] ACK received for 0
[sender] Sent packet seq=4
[sender] Timeout, retransmitting seq=1
[sender] Timeout, retransmitting seq=3
[receiver] Delivered seq=1
[receiver] Delivered seq=2
[receiver] Delivered seq=3
[receiver] Duplicate packet seq=3 (re-ACKed)
[sender] Timeout, retransmitting seq=4
[sender] ACK received for 1
[sender] Sent packet seq=5
[sender] Sent packet seq=6
[sender] ACK received for 3
[sender] Sent packet seq=7
[receiver] Delivered seq=4
[receiver] Delivered seq=5
```

```
[receiver] Delivered seq=6
[receiver] Delivered seq=7
[sender] ACK received for 4
[sender] ACK received for 7
[sender] Timeout, retransmitting seq=5
[sender] Timeout, retransmitting seq=6
[sender] Timeout, retransmitting seq=5
[sender] Timeout, retransmitting seq=6
[receiver] Duplicate packet seq=5 (re-ACKed)
[receiver] Duplicate packet seq=6 (re-ACKed)
[sender] Timeout, retransmitting seq=5
[sender] Timeout, retransmitting seq=6
[receiver] Duplicate packet seq=6 (re-ACKed)
[sender] ACK received for 6
[sender] Timeout, retransmitting seq=5
[receiver] Duplicate packet seq=5 (re-ACKed)
[sender] ACK received for 5
[sender] Sent FIN
[receiver] FIN received, FINACK sent
[sender] Received FINACK - transfer complete

retransmissions = 13 | packets = 8
```

B.4 Raw summary of all experiments (mean ± SD)

Table B.1: Results of all experiments

Exp.	Scenario	Mode	Window	Runs	Throughput (KB/s)	Time (s)	Retrans.
A	S0 (no loss)	SAW	1	10	95.0 ± 0.6	1.053 ± 0.006	0.0 ± 0.0
A	S0 (no loss)	GBN	8	10	726.0 ± 7.6	0.138 ± 0.001	0.0 ± 0.0
A	S0 (no loss)	SR	8	10	725.2 ± 6.1	0.138 ± 0.001	0.0 ± 0.0
A	S1 (low loss)	SAW	1	10	63.0 ± 9.1	1.613 ± 0.201	9.3 ± 3.4
A	S1 (low loss)	GBN	8	10	230.2 ± 94.7	0.487 ± 0.159	39.3 ± 19.5
A	S1 (low loss)	SR	8	10	213.1 ± 67.6	0.500 ± 0.110	9.3 ± 3.4
A	S2 (medium loss)	SAW	1	10	42.5 ± 4.8	2.380 ± 0.264	22.0 ± 4.4
A	S2 (medium loss)	GBN	8	10	117.0 ± 34.7	0.918 ± 0.252	90.6 ± 32.7
A	S2 (medium loss)	SR	8	10	129.7 ± 23.0	0.790 ± 0.123	22.0 ± 4.4
A	S3 (high loss)	SAW	1	10	23.7 ± 3.0	4.285 ± 0.622	53.5 ± 10.3
A	S3 (high loss)	GBN	8	10	57.6 ± 9.0	1.779 ± 0.306	187.9 ± 35.6
A	S3 (high loss)	SR	8	10	72.4 ± 11.3	1.409 ± 0.205	53.5 ± 10.3
B	window 1	GBN	1	5	39.9 ± 3.3	2.517 ± 0.192	24.2 ± 3.3
B	window 1	SR	1	5	40.1 ± 3.4	2.508 ± 0.199	24.2 ± 3.3
B	window 2	GBN	2	5	65.3 ± 14.9	1.587 ± 0.312	32.8 ± 9.8
B	window 2	SR	2	5	58.2 ± 8.6	1.747 ± 0.244	24.2 ± 3.3
B	window 4	GBN	4	5	96.6 ± 35.6	1.156 ± 0.444	55.0 ± 27.4
B	window 4	SR	4	5	78.9 ± 12.9	1.296 ± 0.220	24.2 ± 3.3
B	window 8	GBN	8	5	113.5 ± 42.1	0.966 ± 0.303	97.8 ± 37.2
B	window 8	SR	8	5	128.0 ± 31.2	0.813 ± 0.161	24.2 ± 3.3
B	window 16	GBN	16	5	132.7 ± 71.8	0.890 ± 0.326	185.6 ± 66.3
B	window 16	SR	16	5	190.9 ± 47.6	0.546 ± 0.111	24.2 ± 3.3
B	window 32	GBN	32	5	150.5 ± 21.4	0.676 ± 0.099	269.0 ± 68.7
B	window 32	SR	32	5	268.4 ± 55.6	0.384 ± 0.067	24.2 ± 3.3
C	S4 (5% loss + reorder)	GBN	8	10	19.9 ± 1.5	5.039 ± 0.389	474.6 ± 37.9
C	S4 (5% loss + reorder)	SR	8	10	166.2 ± 30.7	0.617 ± 0.091	9.3 ± 3.4

The per-run values are stored in raw_runs.csv and the summary in results_stats.csv, produced by benchmark_stats.py.